



INSTITUTO POLITÉCNICO NACIONAL
Escuela Superior de Cómputo
ESCOM

Exposición: Algoritmo de Optimización colonia de hormigas (ACO)

Fecha de Entrega: 26/11/2023

Equipo numero 1

Presenta:

García Cerón Diego

Cesar López Cruz

Pérez Carbajal Ezequiel Nahom

Grupo: 6CV1

Materia:

Algoritmos Bioinspirados | Genetic Algorithms

Profesor:

Rosas Trigueros Jorge Luis

Contenido

Introducción	2
¿Qué es la Biometica?	3
Los principales beneficios de la biométrica y arquitectura son:	3
Inicios de la computación BioInspirada.....	3
El comportamiento de las Colonias de Hormigas:	4
Algoritmos de Optimización basados en Colonias de Hormigas:.....	4
Cómo Funciona ACO:.....	5
Representaciones matemáticas del comportamiento de las hormigas:.....	5
Aplicaciones en el Mundo Real:	7
Ventajas de ACO:.....	8
Desventajas de ACO:	8
Desafíos y Limitaciones:	9
Código Python	10
Conclusión	13
BIBLIOGRAFÍAS	14

Introducción

Como sabemos la finalidad a lo largo de este curso es poder conocer y obtener conocimientos de comportamientos que existen dentro de la naturaleza que resultan ser útiles y que tienen la capacidad de adquirir un resultado más “optimo”, que toman demasiada relevancia en el mundo moderno, gracias a la limitación de recursos, por lo que, que mejor maestro del cual aprender que la misma naturaleza, ya que al ser la misma naturaleza el habitante más antiguo de este mundo y los mismos seres vivos que cuentan con la habilidad de adaptación bajo diferentes ecosistemas, por lo que aprender estas técnicas y adaptarlas a nuestros problemas actuales puede ser la clave para el futuro, es por eso que nuestra exposición se basa en la forma de organización de las hormigas al momento de hallar alimentos y hallar rutas más seguras y más cortas contando con un conocimiento colectivo adaptado en diferentes entornos y bajo diferentes circunstancias.

Algoritmo de Optimización Colonia de hormigas (Ant Colony Optimization ACO)

¿Qué es la Biometica?

Biomimética hace significado a Imitar la vida.

Proviene de “Bios” que es Vida y “Mimesis” que es Imitar.

El concepto de biomimetismo es cómo podemos estudiar las mejores ideas del mundo natural para poder imitar sus diseños, organización y procesos para solucionar problemáticas del ser humano en la actualidad.

No se trata de copiar a la naturaleza sino de emularla e inspirarse.

Se basa en preguntarse ¿Qué haría la naturaleza en una situación concreta?

Durante siglos, aunque especialmente las últimas décadas, el ser humano se ha intentado imponer a la naturaleza, la hemos forzado y exprimido todo lo que podíamos aprovechar de ella. Y luego le hemos devuelto lo que no nos interesaba.

Aquí una pregunta:

¿Y si en lugar de esta actitud de dominación hacia ella, intentamos comprenderla y aprender de ella?

Los principales beneficios de la biomética y arquitectura son:

- Baja o casi nula producción de residuos
- Incremento del ahorro de energía.
- Mejoras de la eficiencia (energía, construcción y materiales)

Inicios de la computación BioInspirada

Los algoritmos bioinspirados comenzaron a desarrollarse y a ganar popularidad principalmente a partir de la década de 1990.

Los sistemas bioinspirados son sistemas contruidos por medio de hardware configurables y sistemas electrónicos que emulan la forma de pensar, el modo de procesar información y resolución de problemas de los sistemas biológicos.

Los desarrollos iniciales en este campo se centraron en el uso y desarrollo de aproximaciones evolutivas para la obtención automática de controladores para robots autónomos.

Además de la aplicación de algoritmos evolutivos para la resolución de problemas en áreas tan diversas como el diseño naval o la logística.

Los resultados de esta área permiten obtener un mayor conocimiento de los algoritmos y los problemas de forma que la aplicación de uno u otro algoritmo evolutivo se dirija a obtener las mejores soluciones posibles en cada caso.

Dentro de algunos de los primeros programas de optimización y de los mas populares en su inicio fue el desarrollo de:

Algoritmos de Colonias de Hormigas (ACO): Propuestos por Marco Dorigo a principios de la década de 1990, se inspiran en el comportamiento de las hormigas reales para resolver problemas de optimización.

El comportamiento de las Colonias de Hormigas:

Puntos clave:

En nuestro mundo natural, las hormigas (inicialmente) vagan de manera aleatoria, al azar, y una vez encontrada comida regresan a su colonia dejando un rastro de feromonas. Si otras hormigas encuentran dicho rastro, es probable que estas no sigan caminando aleatoriamente, puede que estas sigan el rastro de feromonas, regresando y reforzándolo si estas encuentran comida finalmente.

problemática:

Sin embargo, al paso del tiempo el rastro de feromonas comienza a evaporarse, reduciéndose así su fuerza de atracción. Cuanto más tiempo le tome a una hormiga viajar por el camino y regresar de vuelta otra vez, más tiempo tienen las feromonas para evaporarse.

Un camino corto, en comparación, es marchado más frecuentemente, y por lo tanto la densidad de feromonas se hace más grande en caminos cortos que en los largos. La evaporación de feromonas también tiene la ventaja de evitar convergencias a óptimos locales. Si no hubiese evaporación en absoluto, los caminos elegidos por la primera hormiga tenderían a ser excesivamente atractivos para las siguientes hormigas. En este caso, el espacio de búsqueda de soluciones sería limitado.

Por tanto, cuando una hormiga encuentra un buen camino entre la colonia y la fuente de comida, hay más posibilidades de que otras hormigas sigan este camino y con una retroalimentación positiva se conduce finalmente a todas las hormigas a un solo camino.

Algoritmos de Optimización basados en Colonias de Hormigas:

La idea del algoritmo colonia de hormigas es imitar este comportamiento con "hormigas simuladas" caminando a través de un grafo que representa el problema en cuestión.

En ciencias de la computación y en investigación operativa, el algoritmo de optimización de la colonia de hormigas (Ant Colony Optimization, ACO) es una técnica probabilística para solucionar problemas computacionales que pueden reducirse a buscar los mejores caminos o rutas en grafos.

Este algoritmo es un miembro de la familia de los algoritmos de colonia de hormigas, dentro de los métodos de inteligencia de enjambres. Inicialmente propuesto por Marco Dorigo en

1992 en su tesis de doctorado, el primer algoritmo surgió como método para buscar el camino óptimo en un grafo, basado en el comportamiento de las hormigas cuando estas están buscando un camino entre la colonia y una fuente de alimentos. La idea original se ha diversificado para resolver una amplia clase de problemas numéricos, y como resultado, han surgido gran cantidad de problemas nuevos, basándose en diversos aspectos del comportamiento de las hormigas.

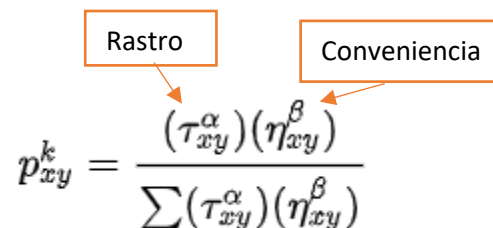
Cómo Funciona ACO:

El algoritmo de optimización de colonia de hormigas replica este comportamiento natural que tienen las hormigas mediante una serie de pasos:

- ❖ **Inicialización de feromonas:** Se asigna una cantidad de feromona inicial a todas las rutas posibles.
- ❖ **Movimiento de hormigas:** Las hormigas seleccionan rutas basadas en una regla probabilística que tiene en cuenta la cantidad de feromona presente y la distancia recorrida.
- ❖ **Actualización de feromonas:** Después de que todas las hormigas han completado su recorrido, se actualiza la cantidad de feromona en cada camino. Las rutas más cortas tienen una mayor cantidad de feromona.
- ❖ **Evaporación de feromonas:** Para evitar la convergencia prematura hacia una solución subóptima, se simula la evaporación gradual de las feromonas en todas las rutas.
- ❖ **Convergencia hacia una solución:** Con el tiempo, las feromonas acumuladas guían a las hormigas hacia la ruta más corta y eficiente.

Representaciones matemáticas del comportamiento de las hormigas:

La probabilidad de tomar un camino se basa en:


$$p_{xy}^k = \frac{(\tau_{xy}^{\alpha})(\eta_{xy}^{\beta})}{\sum (\tau_{xy}^{\alpha})(\eta_{xy}^{\beta})}$$

Donde:

- ❖ K es una hormiga
- ❖ X y Y son ejes de movimiento

Alfa y beta son parámetros positivos que representan la influencia de la feromona sobre el camino.

- ❖ Si **alfa** es alto se tomará preferencia a los caminos con mayores feromonas.
- ❖ Si **beta** es alto se tomará como preferencia los caminos menos costosos.

El punto de ACO radica en hallar el equilibrio entre estos dos parámetros, rastro y costo.

Actualización de feromonas:

Cuando todas las hormigas han completado una solución, los rastros son actualizados por

$$\tau_{xy} \leftarrow (1 - \rho)\tau_{xy} + \sum_k \Delta\tau_{xy}^k$$

Donde:

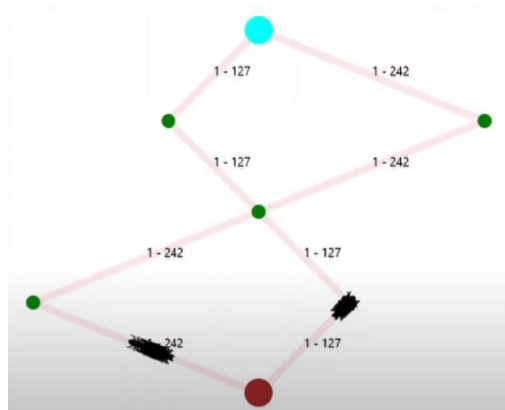
- ❖ “TXY” es la cantidad de feromonas depositadas para un estado de transición XY
- ❖ ρ es el coeficiente de evaporación de feromonas
- ❖ $\Delta\tau_{xy}^k$ es la cantidad de feromonas depositadas por la Kth hormiga

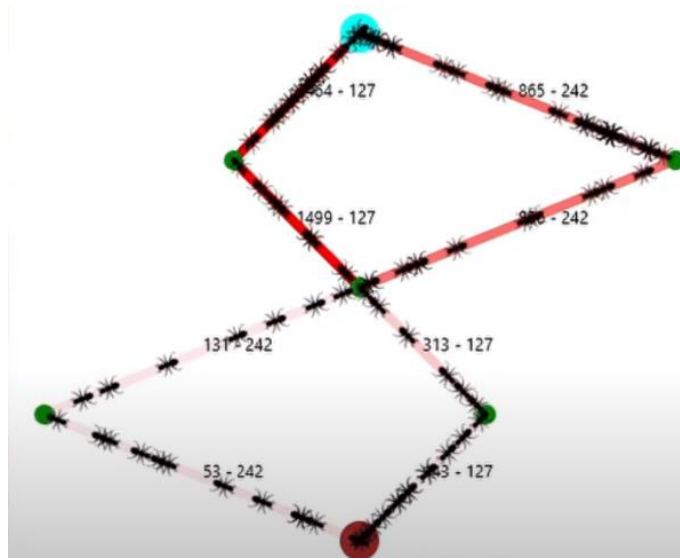
Típicamente dado para el Problema del Viajante (con movimientos correspondientes a los arcos del grafo) dada por:

$$\Delta\tau_{xy}^k = \begin{cases} Q/L_k & \text{if ant } k \text{ uses curve } xy \text{ in its tour} \\ 0 & \text{otherwise} \end{cases}$$

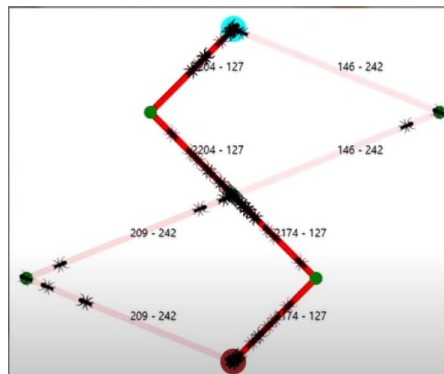
Donde:

- ❖ LK es el costo de la ruta de la Kth hormiga (en la mayoría de los casos es la longitud)
- ❖ Q es una constante.





En algunas ocasiones puede resultar que no se halle el camino más óptimo y que se elija ir por un camino más largo, esto como se puede evitar, bien para solucionar este problema común sería el tomar en cuenta que el rastro de feromona debe de tener una tasa de decremento, esto quiere decir que entre más tiempo tarde en pasar una nueva hormiga, el rastro de feromona anterior ira perdiendo peso y presencia, por lo que un camino más corto tendrá mayor flujo de hormigas gracias a que al tener un tramo más corto la ida y la regresada de la hormiga ira reforzando el rastro, mientras que al tomar un camino más largo la ida y regresada del rastro de la feromona se hallara con una diferencia muy grande y por lo tanto tendrá menor peso.



Aplicaciones en el Mundo Real:

Su aplicación en el mundo real es importante por su capacidad para encontrar soluciones cercanas a las óptimas en problemas complejos lo hace muy atractivo en el campo de la optimización.

Problema del viajante (TSP): Es una de las aplicaciones más conocidas de ACO. Se utiliza para encontrar la ruta más corta que visite todas las ciudades exactamente una vez y

regrese al punto de partida. Esto tiene aplicaciones en logística, planificación de rutas de entrega y diseño de redes de telecomunicaciones.

Enrutamiento de vehículos: ACO se aplica para optimizar rutas de vehículos de reparto, como camiones de entrega, para minimizar la distancia recorrida y maximizar la eficiencia en la entrega de bienes.

Diseño de redes de comunicación: En la planificación y diseño de redes de telecomunicaciones, ACO puede ayudar a encontrar la disposición óptima de nodos y conexiones para minimizar los costos o maximizar la eficiencia de la red.

Planificación de horarios: En entornos complejos como la programación de horarios escolares o laborales, ACO puede ser utilizado para optimizar la distribución de recursos y tiempos.

Optimización de rutas de cableado y circuitos: En ingeniería eléctrica y electrónica, ACO se usa para diseñar rutas de cableado eficientes y circuitos electrónicos.

Optimización de producción: En la industria manufacturera, ACO se emplea para mejorar el diseño de líneas de producción, minimizar tiempos de espera y maximizar la eficiencia en la distribución de tareas.

Ventajas de ACO:

Capacidad para encontrar soluciones cercanas a óptimas: ACO es conocido por su habilidad para encontrar soluciones cercanas a óptimas en problemas complejos de optimización combinatoria, como el Problema del Viajante (TSP) y problemas de enrutamiento.

Adaptabilidad a diferentes problemas: Puede aplicarse a una amplia gama de problemas de optimización, desde rutas de vehículos hasta diseño de redes, debido a su naturaleza adaptable y versátil.

Paralelización: Es posible paralelizar el algoritmo, lo que significa que puede manejar eficientemente problemas grandes al distribuir la búsqueda entre múltiples agentes (hormigas).

Inspirado en la naturaleza: Está inspirado en el comportamiento de las hormigas reales y su capacidad para encontrar caminos eficientes. Esta inspiración biológica a menudo conduce a soluciones innovadoras y efectivas.

Desventajas de ACO:

Sensibilidad a parámetros: ACO tiene parámetros sensibles que deben ser ajustados correctamente para obtener un rendimiento óptimo. Modificar estos parámetros puede ser un desafío y podría afectar la convergencia del algoritmo.

Complejidad computacional: En problemas grandes y complejos, la cantidad de feromonas y caminos posibles puede aumentar significativamente, lo que conlleva un aumento en la complejidad computacional y el tiempo de ejecución.

Convergencia lenta: En algunos casos, ACO puede tener una convergencia lenta hacia soluciones óptimas, especialmente en problemas con espacios de búsqueda grandes o mal condicionados.

Dependencia de la heurística: La eficacia de ACO puede depender de la heurística utilizada para guiar la exploración de las hormigas, y la elección de esta heurística puede no ser siempre obvia o fácil de determinar.

Desafíos y Limitaciones:

Eficiencia computacional:

Manejo de grandes conjuntos de datos: ACO puede volverse computacionalmente costoso a medida que aumenta el tamaño del problema. Desarrollar estrategias para hacer frente a la complejidad computacional en problemas grandes es crucial para su aplicación práctica.

Mejora de la convergencia:

Convergencia rápida hacia soluciones óptimas: Lograr una convergencia más rápida y eficiente hacia soluciones óptimas o cercanas a óptimas es un desafío importante. Estrategias para mejorar la exploración del espacio de búsqueda y evitar la convergencia prematura hacia soluciones subóptimas son necesarias.

Adaptación a problemas dinámicos:

Manejo de cambios en tiempo real: La capacidad de adaptarse y encontrar soluciones eficientes en entornos dinámicos o con cambios en los datos o restricciones es esencial. Hacer que el algoritmo sea más adaptable a cambios y actualizaciones continuas en tiempo real es un desafío significativo.

Parametrización y heurísticas:

Selección adecuada de parámetros y heurísticas: La configuración de parámetros y la elección de heurísticas adecuadas para guiar la búsqueda son aspectos críticos. Desarrollar métodos automáticos o más robustos para la configuración de parámetros y heurísticas es un área de mejora.

Robustez y generalización:

Generalización a diferentes dominios y problemas: ACO puede ser sensible a las características específicas de ciertos problemas. Mejorar su robustez y capacidad para generalizarse a una variedad más amplia de problemas es un desafío clave para su aplicación en diversos campos.

Exploración y explotación del espacio de búsqueda:

Equilibrio entre exploración y explotación: Encontrar un equilibrio entre la exploración del espacio de búsqueda para encontrar nuevas soluciones y la explotación de las soluciones existentes para mejorar la calidad de las soluciones es un desafío importante.

Adaptabilidad y escalabilidad:

Adaptación a la escalabilidad: Hacer que el algoritmo sea más adaptable y escalable para manejar problemas de gran escala es fundamental. Mejorar su capacidad para abordar problemas complejos y grandes de manera eficiente es un desafío continuo.

Código Python

```
import random
import networkx as nx
import matplotlib.pyplot as plt

class ColoniaHormigas:
    def __init__(self, grafo, n_hormigas=10, alfa=1.0, beta=2.0,
evaporacion=0.5, Q=1.0):
        self.grafo = grafo
        self.n_hormigas = n_hormigas
        self.alfa = alfa
        self.beta = beta
        self.evaporacion = evaporacion
        self.Q = Q
        self.feromonas = self.inicializar_feromonas()

    def inicializar_feromonas(self):
        feromonas = {}
        for origen in self.grafo:
            for destino in self.grafo[origen]:
                feromonas[(origen, destino)] = 1.0
        return feromonas

    def elegir_siguiente_nodo(self, nodo_actual, camino):
        vecinos = [nodo for nodo in self.grafo[nodo_actual] if nodo
not in camino]
        if not vecinos:
            return camino[0] # Regresar al origen si no hay vecinos
no visitados

        probabilidades = self.calcular_probabilidades(nodo_actual,
vecinos)
        return random.choices(vecinos, weights=probabilidades)[0]

    def calcular_probabilidades(self, nodo_actual, vecinos):
        probabilidades = []
        for vecino in vecinos:
            # Asegurarse de que la arista esté presente en el
diccionario
            arista = (nodo_actual, vecino)
            if arista in self.feromonas:
                feromona = self.feromonas[arista]
                distancia = self.grafo[nodo_actual][vecino]
                probabilidad = feromona**self.alfa * (1 /
distancia)**self.beta
```

```

        probabilidades.append(probabilidad)
    else:
        # Si la arista no está en el diccionario, asignar una
        # probabilidad mínima
        probabilidades.append(0.0001)
    suma_probabilidades = sum(probabilidades)
    return [p / suma_probabilidades for p in probabilidades]

def encontrar_caminos(self, origen, destino):
    todos_los_caminos = []
    for _ in range(self.n_hormigas):
        camino = self.generar_camino(origen, destino)
        todos_los_caminos.append((camino,
self.calcular_costo_camino(camino)))
    return todos_los_caminos

def generar_camino(self, origen, destino):
    camino = [origen]
    while camino[-1] != destino:
        siguiente_nodo = self.elegir_siguiente_nodo(camino[-1],
camino)
        camino.append(siguiente_nodo)
    return camino

def calcular_costo_camino(self, camino):
    costo = 0
    for i in range(len(camino) - 1):
        costo += self.grafo[camino[i]][camino[i + 1]]
    return costo

def actualizar_feromonas(self):
    for clave in self.feromonas:
        self.feromonas[clave] *= (1 - self.evaporacion)
    for camino, costo in self.todas_los_caminos:
        longitud_camino = len(camino)
        for i in range(longitud_camino - 1):
            arista = (camino[i], camino[i + 1])
            self.feromonas[arista] += self.Q / costo

        # Asegurarse de que la arista en la dirección opuesta
        # también tenga una entrada en el diccionario
        arista_inversa = (camino[i + 1], camino[i])
        if arista_inversa not in self.feromonas:
            self.feromonas[arista_inversa] = 1.0

```

```

def ejecutar(self, origen, destino, iteraciones=1):
    for _ in range(iteraciones):
        self.todas_los_caminos = self.encontrar_caminos(origen,
destino)

        self.actualizar_feromonas()
    return min(self.todas_los_caminos, key=lambda x: x[1])
def visualizar_grafo(self, mejor_camino=None):
    G = nx.Graph(self.grafo)

    pos = nx.spring_layout(G)

    # Dibujar nodos
    nx.draw_networkx_nodes(G, pos, node_size=700,
node_color='skyblue')

    # Dibujar aristas con etiquetas de costos
    edge_labels = {(i, j): str(self.grafo[i][j]) for i, j in
G.edges()}
    nx.draw_networkx_edges(G, pos, width=2, edge_color='gray')
    nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)

    # Dibujar etiquetas de nodos
    nx.draw_networkx_labels(G, pos, font_size=8,
font_color='black', font_weight='bold')

    # Resaltar mejor camino en rojo
    if mejor_camino:
        mejor_camino_edges = [(mejor_camino[i], mejor_camino[i +
1]) for i in range(len(mejor_camino) - 1)]
        nx.draw_networkx_edges(G, pos,
edgelist=mejor_camino_edges, edge_color='red', width=2)

    plt.show()

# Ejemplo de uso:
grafo = {
    'A': {'B': 1, 'C': 2, 'D': 3},
    'B': {'A': 1, 'C': 2, 'D': 2, 'E': 3},
    'C': {'A': 2, 'B': 2, 'D': 1},
    'D': {'A': 3, 'B': 2, 'C': 1, 'E': 2},
    'E': {'B': 3, 'D': 2}
}

colonia = ColoniaHormigas(grafo, n_hormigas=3)

```

```

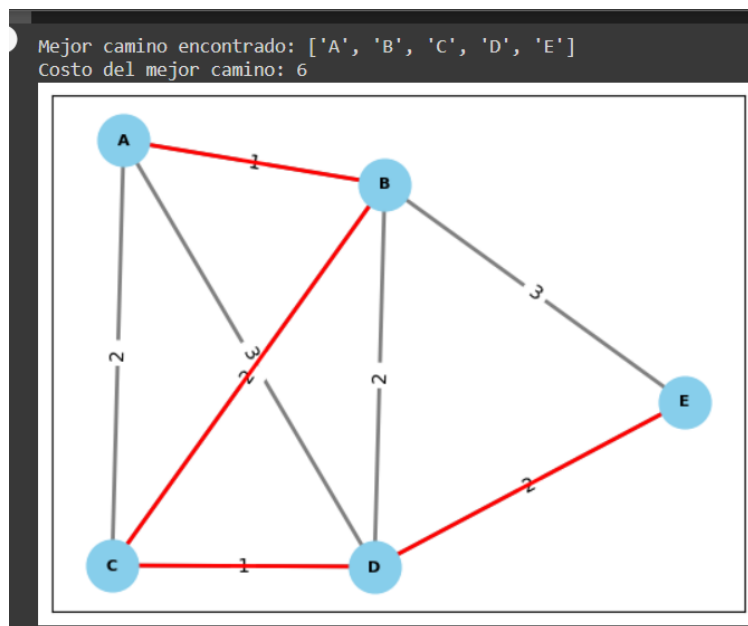
mejor_camino, mejor_costo = colonia.ejecutar('A', 'E', iteraciones=1)
#Ejemplo de uso 2:
#grafo = {
# 'A': {'B': 1, 'C': 2, 'D': 3},
# 'B': {'A': 1, 'C': 2, 'D': 2},
# 'C': {'A': 2, 'B': 2, 'D': 1},
# 'D': {'A': 3, 'B': 2, 'C': 1},
#}

#colonia = ColoniaHormigas(grafo, n_hormigas=50)
#mejor_camino, mejor_costo = colonia.ejecutar('A', 'D',
iteraciones=100)

print("Mejor camino encontrado:", mejor_camino)
print("Costo del mejor camino:", mejor_costo)

colonia.visualizar_grafo(mejor_camino)

```



Conclusión

Los algoritmos bioinspirados comenzaron a desarrollarse y a ganar popularidad principalmente a partir de la década de 1990 y a medida que avanzaba la investigación en inteligencia artificial y ciencias de la computación, se profundizó en la aplicación de estos algoritmos bioinspirados para resolver problemas complejos en campos como la optimización, el aprendizaje automático, la robótica, entre otros. Desde entonces, la investigación en algoritmos bioinspirados ha seguido creciendo y expandiéndose, encontrando aplicaciones en una variedad cada vez mayor de áreas.

BIBLIOGRAFIAS

<https://www.simbiotia.com/biomimesis/>

<https://www.gii.udc.es/presentacion>

https://es.wikipedia.org/wiki/Sistemas_bioinspirados

<https://www.youtube.com/watch?v=xauOQcvLW3c&t=4s>

<https://www.youtube.com/watch?v=knYGNpiXqLc&t=344s>